AI Repetitor

Architecture & Under-the-Hood Guide — for demo prep

`Next.js 16 · Google Gemini · Vercel Serverless`

---

1. What the app does

AI Repetitor is an AI-powered study tool. A student uploads a lecture slide deck as a PDF. The app narrates it slide-by-slide out loud, in Azerbaijani, generated fresh by an AI model that actually looks at each slide image — not a canned script. While listening, the student can interrupt at any point and ask a question, by typing or by voice, and the app answers out loud, highlights the exact term/number/formula it's talking about directly on the slide image, and — if the answer is really about a different slide — automatically jumps there to show it, then returns to where the student left off once they manually ask to continue.

2. Tech stack at a glance

Layer	Choice	Why
Framework	Next.js 16 (App Router, TypeScript)	One codebase for UI + serverless API routes; deploys natively to Vercel
Hosting	Vercel (serverless functions)	Required target — rules out anything needing a persistent server or native binaries
AI provider	Google Gemini (`@google/genai` SDK)	One provider for vision, text, and native speech synthesis
PDF rendering	`pdfjs-dist` , entirely client-side (in the browser)	Vercel has no server-side native rendering (e.g. LibreOffice) — the browser does the rendering instead
Server state	TanStack Query	Per-slide narration/TTS results are cached and fetched independently, with built-in loading/error states
UI state	Plain React state + custom hooks	Small enough app that a global store (Redux/Zustand) would add ceremony with no benefit
Styling	Tailwind CSS v4 + CSS custom properties	Fast to iterate on a custom "notebook" visual identity

3. High-level architecture

Browser — renders PDF pages to images (pdf.js), plays audio, drives the UI, holds all session state

⇅ JSON over HTTP (one route streams plain text)

Next.js Route Handlers (serverless functions, `maxDuration=30s` each) — `/api/ingest` , `/api/tts` , `/api/transcribe` , `/api/chat` , `/api/chat/focus`

⇅ HTTPS via @google/genai SDK

Google Gemini API — vision reasoning, text reasoning, and native text-to-speech

There is no database and no persistent server. A deck's slides live only in the browser's memory for that session — re-uploading the same PDF re-generates everything. This keeps the MVP simple; nothing about the architecture would prevent adding persistence later.

4. Ingestion pipeline: PDF → narrated slides

Uploading a PDF (`components/UploadZone.tsx` → `hooks/usePdfUpload.ts`) kicks off, per slide:

1. **pdf.js renders the page to a JPEG** in a `<canvas>` , entirely in the browser (`lib/pdfRender.ts`) — resolution capped so the longer edge is ≤1280px, quality 0.85

↓

2. **pdf.js also extracts the text layer** — every text run's exact position on the page, as 0-1 fractions of page width/height (resolution-independent)

↓

3. The JPEG is POSTed to `/api/ingest` → Gemini vision (`gemini-flash-latest`) writes a 2-4 sentence Azerbaijani narration *and* an ordered list of the key terms it mentions

↓

4. The narration text is POSTed to `/api/tts` → Gemini's native TTS (`gemini-2.5-flash-preview-tts` , voice "Kore") returns raw PCM audio

↓

5. Server wraps the PCM in a WAV header (`lib/wav.ts`) → result cached per-slide (TanStack Query) — ready to auto-play

Why PDF only, and why client-side rendering: the brief called for real slide visuals (not reconstructed text) while staying deployable on Vercel with no native server dependencies. Server-side rendering of PPTX/PDF normally needs LibreOffice or a native canvas library — not viable on serverless. Doing the rasterization in the browser with `pdfjs-dist` sidesteps that entirely, at the cost of supporting PDF only (not PPTX) for the MVP.

Concurrency: Gemini's free tier allows only a handful of requests per minute, so slides aren't all sent at once. A small semaphore (`lib/semaphore.ts`) caps ingestion to 2 slide-chains running concurrently (`hooks/useIngestSlides.ts`), each chain being the sequential narrate-then-speak pair above — slide 1 becomes playable the moment its own two calls finish, without waiting for the rest of the deck.

5. The two AI models, and why

Model	Used for	Notes
`gemini-flash-latest` (alias — currently resolves to Gemini 3.6 Flash)	Slide narration (vision input), chat answers (streamed), voice transcription, focus-term/relevant-slide extraction	One general text+vision model reused everywhere, with structured JSON output (`responseSchema`) for anything the app needs to parse rather than just display
`gemini-2.5-flash-preview-tts`	Turning narration/answer text into spoken audio	Google's native speech model. Voice: **"Kore"**. Returns raw headerless PCM audio (`audio/L16` , 24kHz, 16-bit, mono), which the server wraps in a minimal 44-byte WAV header (`lib/wav.ts`) before sending to the browser, since browsers can't play raw PCM directly

Why an alias, not a pinned version: `gemini-2.5-flash` itself was found to return a 404 for new API keys mid-project even though it's still listed by the API — Gemini model availability shifts quickly. Aliases like `-latest` auto-track Google's current recommended model, reducing that risk.

A single shared system prompt (`AZERBAIJANI_SYSTEM_PROMPT` in `lib/gemini/client.ts`) instructs every text/vision call to answer only in Azerbaijani, regardless of the source PDF's language — so the "respond only in Azerbaijani" rule is defined once and can't drift between features. Transcription uses a separate, more specific instruction, because speech-to-text has a distinct failure mode: Azerbaijani and Turkish are close enough that models tend to "correct" Azerbaijani spelling into Turkish orthography unless explicitly told not to.

> **Transcription accuracy improvement:** the transcription call now also receives the current slide's narration and a deck summary as a *vocabulary hint* (`contextText` in `TranscribeRequest`) — not to translate or comment on, purely so lecture-specific Azerbaijani terms and proper nouns are recognized correctly instead of guessed at cold from audio alone. No extra API call — folded into the existing transcription request.

6. On-slide highlighting & narration sync

A simplified "focus" feature was the original spec (bolding a term in the chat reply). This was extended to a real, positioned highlight box drawn directly on the slide image:

- **Position data** comes from the same pdf.js text-layer extraction described in §4 — each text run's left/top/width/height, as fractions of the page.
- `lib/findFocusHighlight.ts` takes a term (e.g. "Şümüllər sistemi") and finds where it appears in that slide's extracted text, returning a bounding box (the union of every text run it spans). If it isn't found — e.g. the model paraphrased instead of quoting — nothing extra is drawn; the chat bubble's bold highlight is always the fallback.

- **During plain narration playback** (not just chat), the highlight moves between several terms over time: Gemini also returns an *ordered list* of key terms per slide (§4, step 3), and each term's approximate position within the narration is computed as `text.indexOf(term) / text.length`. The `<audio>` element's `timeupdate` event is compared against those fractions (`hooks/useSlideAudioPlayer.ts`) to pick whichever term playback has reached.

> This is a deliberate approximation, not real word-level audio sync — Gemini's TTS gives no timestamps to sync against. It was confirmed with the intended behavior ("multiple highlights that change over time" vs. one static highlight) before building it this way.

7. The chat feature

The chat panel is a normal message thread. A question can be composed two ways, and both land in the exact same place:

Type directly into the input box	**Hold the mic** → recorded audio → `/api/transcribe` → text appears in the input box for the student to review/edit before sending

Nothing is ever auto-sent straight from voice — the transcript always lands in the editable input first. Once sent, narration is paused immediately (so it doesn't talk over the answer) and stays paused — see §11 for exactly how and why it never auto-resumes.

8. Streaming the chat answer

Originally, one combined Gemini call produced the answer text *and* the structured `focusTerm` / `relevantSlideNumber` fields together, so the chat bubble sat blank until the entire answer was ready — Gemini generation is the whole latency budget here (server-side JSON parsing/WAV-wrapping runs in single-digit milliseconds by comparison), so a multi-second wait with nothing on screen felt slow. This was split into two calls:

1. `/api/chat` streams back *just the answer text*, plain text chunks, no structured schema — the chat bubble grows on screen as tokens arrive (`lib/gemini/chat.ts` : `streamChatAnswer`, using `generateContentStream`)

↓ once the full text is known

2a. `/api/chat/focus` — a small, fast follow-up call extracts `focusTerm` + `relevantSlideNumber` from the now-complete text	2b. `/api/tts` — speaks the complete answer

Both fire together via `Promise.all` — since TTS only ever needed the full text anyway, splitting off the focus-info call adds no extra delay before audio starts

Why not stream everything, or stream the audio too? Two things were deliberately ruled out:

- **Streaming structured JSON** (asking Gemini to stream `{answerText, focusTerm, ...}` directly) only delivers partial JSON fragments — it would need an incremental JSON parser to pull one field out early, for real added complexity and uncertain gain over the two-call split.
- **True low-latency streaming audio** (sub-100ms first sound) needs Google's Live API — a persistent, stateful WebSocket connection built for live back-and-forth conversation. That's architecturally incompatible with this app's stateless Vercel functions and its "pre-generate every slide upfront" ingestion model. TTS stays one plain call.

> **The remaining silent gap, closed:** even after streaming, there was still an unindicated wait between the full answer text appearing and audio actually starting — TTS generation alone regularly takes 8-14 seconds. A second, distinct indicator ("Səs hazırlanır" — preparing audio) now covers exactly that window, so the UI never goes quiet without explanation.

> Net effect: total time-to-audio is unchanged from before — the same work happens, just reordered. What changed is that the student now sees the answer appear progressively instead of staring at a "Düşünürəm..." (Thinking...) indicator for several seconds, and the handoff to audio is now also indicated instead of silent.

9. Request/response reference — every endpoint

The complete picture of what each server route does, whether it streams, and what model backs it:

Route	Streaming?	Request body	Response	Model(s)
`/api/ingest`	No	slide image (base64 `image/jpeg`), slide index, total slides	JSON: `{ narrationText, focusPoints[] }`	`gemini-flash-latest` (vision, structured JSON)
`/api/tts`	No	text to speak (plain string)	JSON: `{ audioBase64 }` (WAV, server-wrapped)	`gemini-2.5-flash-preview-tts`
`/api/transcribe`	No	recorded audio (base64 `audio/webm;codecs=opus`) + optional vocabulary-hint context text	JSON: `{ text }`	`gemini-flash-latest` (audio input, temperature 0)
`/api/chat`	Yes — plain text chunks	question text, current slide narration, deck summary	`ReadableStream`, `Content-Type: text/plain` — no JSON wrapper	`gemini-flash-latest` (`generateContentStream`, no schema)
`/api/chat/focus`	No	question text, the now-complete answer text, current slide narration, deck summary	JSON: `{ focusTerm, relevantSlideNumber }`	`gemini-flash-latest` (structured JSON)

`/api/chat` **is the only streaming endpoint in the app.** Everything else — ingestion, TTS, transcription, and the focus-info extraction — is a single request/response, since none of those have a UI element that benefits from partial results the way a growing chat bubble does.

10. Performance & scaling choices

Concern	What's done	Why
Image payload size	Slide images capped at 1280px on the longer edge, JPEG quality 0.85 (`lib/pdfRender.ts`)	Keeps the base64 request body small for faster upload and to stay well under Vercel's request-size limits, while remaining sharp enough for Gemini vision to read on-slide text/formulas accurately
Ingestion concurrency	2 concurrent narrate→TTS chains, enforced by a semaphore, not unlimited parallelism	Free-tier Gemini RPM is as low as 3-5 requests/minute in practice — higher concurrency just produces more immediate 429s rather than finishing faster
Chat answer latency (perceived)	Streamed text (§8) + a distinct "preparing audio" indicator for the post-text TTS wait	Total backend time is unchanged — this is entirely about not leaving the UI silent during real, unavoidable generation time
Retry timing budget	Retries capped so they can never exceed the 30s route `maxDuration` (§12)	A retry that risks the platform killing the function returns nothing useful — failing fast with an honest wait-time message is strictly better
Per-slide caching	TanStack Query, `staleTime: Infinity` per slide	A slide's narration/audio never changes once generated in a session — no reason to ever refetch it
Audio replay cost	One `<audio>` element cached per slide index for the deck's lifetime, not recreated on every pause/resume or slide revisit	Avoids redundant decode work and — critically — is what makes resuming from the exact paused position possible at all (§11)

11. Cross-slide navigation & pause/resume

If a question is really about a different slide than the one currently showing (e.g. "which slide had the diagram of the digestive system?"), the app jumps there automatically:

- The chat call is given a short *summary of every slide* (each slide's narration, labelled "Slayd 1", "Slayd 2", ...) as extra context, so the model already knows what each slide contains.
- Gemini directly names **which slide number** the answer actually concerns (`relevantSlideNumber`), rather than the app trying to text-search for a match — critical for slides where the relevant content is a photo/diagram with no matching extractable text at all.
- The app jumps there without playing narration — it stays paused regardless.

> **Narration never auto-resumes, ever — by design.** Whether a question was about the current slide or a different one, whether the answer played successfully or failed entirely, narration always stays paused after a chat interaction. The only way it starts again is the student explicitly pressing the pause/resume button — which, when a cross-slide jump happened, reads "Slayd N-ə qayıt" (Return to slide N) and both returns *and* resumes in one click.

> **Resuming continues from the exact paused position, not the beginning.** Each slide's audio element is cached for the whole deck's lifetime (§10) instead of being recreated every time that slide becomes current. This is what lets a mid-narration pause — whether manual, or because a question was asked and the view jumped elsewhere and back — pick up exactly where it left off. The one exception: a slide whose narration already played to completion resets itself back to the start the moment it naturally ends, so replaying a *finished* slide still starts over, as expected.

> This — stay paused unconditionally, resume only on a deliberate click, and resume from position rather than from zero — was a specific, confirmed product decision, not the simplest default.

12. Reliability: rate limits & retries

Google's free tier is aggressively rate-limited, and different failures need different handling — retrying blindly with a fixed backoff either gives up too early or wastes time retrying something retrying can't fix. `lib/retry.ts` distinguishes:

Error	Meaning	Handling
429, per-minute quota	Too many requests this minute	Retry once, waiting exactly as long as Google's own error says to (parsed from the error body), only if that wait fits comfortably inside the route's time limit
429, per-day quota	Daily allowance used up	Never retried — surfaced immediately with an honest message, since waiting can't help within the same day
503, model overloaded	Transient capacity issue on Google's side	Retried up to twice with a short fixed delay — usually clears within seconds
Network blip	Connection reset/timeout	Retried once after a short delay

Every user-facing failure message is specific and in Azerbaijani, centralized in `lib/messages.ts` and applied via `lib/describeApiError.ts` — "wait ~30 seconds" reads very differently from "daily limit reached, try again tomorrow or enable billing," and a generic "try again" would be actively misleading for the second case.

13. Code organization & state management

- `lib/gemini/` — one file per Gemini capability (`narration.ts` , `speech.ts` , `transcription.ts` , `chat.ts`) sharing one client + system prompt from `client.ts` .
- `lib/constants.ts` — every small shared magic value: mime types, JPEG quality, and the `SLIDE_STATUS` / `UPLOAD_STATUS` enums (with their derived TypeScript types), so a status like "ready" or "narrating" is a named constant everywhere it's checked, not a scattered string literal.
- `lib/messages.ts` — every user-facing Azerbaijani error/toast string, grouped by feature area, instead of duplicated inline across routes and hooks.
- `lib/deckSummary.ts` , `lib/audioDataUri.ts` — small, pure, previously-duplicated logic pulled into single-purpose modules.
- **Components are presentational only** — all logic (recording, uploading, sending, playing audio, slide navigation/focus orchestration) lives in dedicated hooks (`useVoiceRecorder` , `usePdfUpload` , `useChatThread` , `useSlideAudioPlayer` , `useIngestSlides` , `usePresentationSession`). A component consumes a hook and renders; it doesn't call into `lib/` domain logic or hold business-logic state itself.
- `hooks/usePresentationSession.ts` — owns everything about "what slide are we on, what's highlighted, where do we return to" (previously living directly in the page component): wraps `useSlideAudioPlayer` , computes the on-slide highlight box, and handles cross-slide jump/return logic. The page component itself is now pure composition — it wires this hook's output to the presentational components and contains no logic of its own.
- **TanStack Query** owns all server/async state (per-slide narration+TTS results, the chat answer mutation) — with `staleTime: Infinity` for slides, since a slide's content never changes once generated.
- **Plain React state** handles UI-only concerns (input text, which slide is showing, pause state) — no global store; the app isn't large enough to need one.

14. Deployment constraints

Every Route Handler sets `maxDuration = 30` (seconds) — Vercel serverless functions are time-boxed, which is precisely why the retry logic above caps how long it will wait before giving up rather than risking the whole platform killing the function mid-retry. There is no persistent server process, no filesystem to write to, and no native dependencies — every design choice above (client-side PDF rendering, stateless per-request Gemini calls, no Live API/WebSocket) works within that boundary.

15. Anticipated questions — cheat sheet

"Which AI model does this use?"

Google Gemini, via the official `@google/genai` SDK. One text/vision model (`gemini-flash-latest`) for understanding slides, answering questions, and transcription, and one dedicated speech model (`gemini-2.5-flash-preview-tts`) for narration and answer audio.

"What file formats do you support, and why not PowerPoint?"

PDF only, by deliberate scope choice. Real slide visuals were required, but the app must run on Vercel serverless with no native rendering dependencies — client-side `pdf.js` rasterization satisfies both. PPTX support was explicitly considered and intentionally left out of scope, not overlooked.

"Walk me through what happens when I upload a file."

The browser rasterizes each page to a size/quality-capped JPEG and extracts text positions locally (pdf.js) → each slide's image goes to Gemini vision for a narration + key terms → the narration text goes to Gemini TTS → raw PCM audio is WAV-wrapped server-side → the slide is cached and ready to auto-play. Two slides ingest concurrently at a time. See §4, §9, §10.

"Do you use streaming? Which requests, exactly?"

Only one: `/api/chat` , which streams the answer as plain text chunks. Every other route (ingest, TTS, transcribe, chat-focus) is a single request/response — see the full table in §9. Streaming doesn't reduce total generation time — Gemini's own generation is the bottleneck either way — but makes the answer appear progressively instead of after one long wait, plus a second indicator now covers the remaining silent gap before audio starts. See §8.

"How does the on-slide highlighting actually work — is it a hardcoded box?"

No — it's computed from the PDF's own text layer at render time (pdf.js `getTextContent()`), so a highlighted box's position is derived from where that text run actually sits on the real page, not guessed or hardcoded. See §6.

"What happens if the AI mentions something from a different slide?"

Gemini is given a running summary of every slide up front and directly names which slide number an answer concerns; the app auto-navigates there. Narration never auto-resumes in this app — same-slide or cross-slide, success or failure — only a manual click resumes it, and it resumes from the exact position it was paused at, not from zero. See §11.

"What happens when the API rate-limits you?"

The app distinguishes a short-lived per-minute limit (worth an automatic retry, waiting exactly as long as Google says to) from a daily limit (not worth retrying at all) from a transient "model overloaded" error (worth a couple of quick retries) — each gets a specific, honest message instead of a generic "try again." See §12.

"Why no database / why does re-uploading regenerate everything?"

Deliberate MVP scope cut given the time budget — nothing in the architecture would prevent adding persistence later; slides are simply generated fresh per session today.

"Why not just use one big model call for everything?"

Two reasons recur throughout: (1) structured output plus streaming output don't mix well in one call — Gemini's streaming API only delivers partial JSON fragments, not clean incremental fields — so the chat answer is deliberately two calls; (2) narration and TTS are different Gemini models entirely (vision-capable text model vs. native speech model), so they were always going to be separate calls.

"How do you keep image upload size down for performance?"

Every slide is rasterized client-side to a JPEG capped at 1280px on the longer edge, at 0.85 quality — enough resolution for Gemini vision to read on-slide text accurately, small enough to keep request bodies fast to upload and safely under Vercel's size limits. See §10.